

ICLR 2023 | Graph Neural Networks Aren't Just Fragile - They're Over-Robust

A semantics-aware look at adversarial robustness from TU Munich, and why a large part of what we call GNN robustness is robustness we should not want

The standard story about graph neural networks (GNNs) is that they are dangerously easy to fool. Flip a handful of edges, and a model that looked accurate suddenly changes its mind. A decade of adversarial-robustness research has treated that fragility as the whole problem: the attacker makes a tiny, meaning-preserving change, and the model that keeps its prediction is the robust, trustworthy one. This ICLR 2023 paper from the Technical University of Munich points out that the story quietly assumes something we never actually check on graphs - that the small change really did preserve the node's meaning.

That assumption is doing a lot of work. An adversarial example is only adversarial if it leaves the input's true label untouched; otherwise a changed prediction is just a correct response to a genuinely different input. For images, a human can glance at the perturbed picture and confirm it still shows a cat. Graphs have no such visual check, so the community settled on a proxy: count how many edges were inserted or deleted, and call the perturbation "small" if that count stays under an IO budget. The paper's central observation is that on real graphs, where most nodes have very few neighbors, even a tiny edge budget can rewire a node completely - and once you rewire a low-degree node, its true class may have already changed.

Read that way, a model that stubbornly keeps its old prediction is not being robust in any good sense. It is being wrong about an input whose ground truth has moved. The authors give this failure mode a name - over-robustness - and build the tools to measure it. Their headline finding is uncomfortable: every GNN they test is significantly over-robust, and a surprisingly large share of the robustness we routinely celebrate lies beyond the point where the node's semantics already flipped.

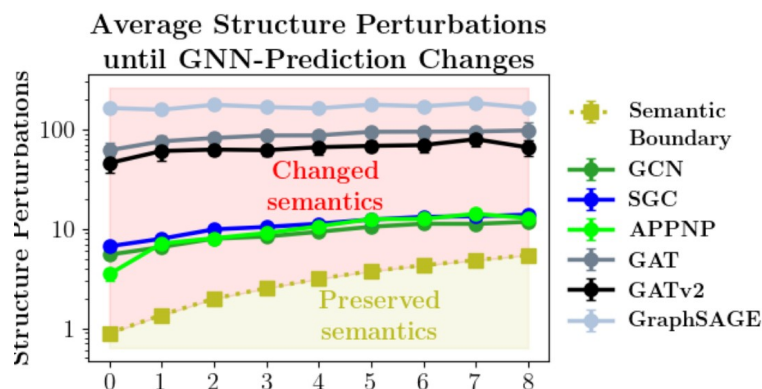


Figure 1. On synthetic CSBM graphs, the dotted "semantic boundary" marks how many edge changes actually flip a node's true (most-likely) class, as a function of node degree. Every GNN (colored curves) only changes its own prediction far above that line - the entire shaded band between the two is robustness the model keeps after the semantics have already changed.

Why "count the edited edges" is not enough

The reason graphs are hard is that you cannot inspect a perturbed node the way you inspect a perturbed image. Prior work tried to make edge-counting more principled by adding proxies - keep the degree distribution roughly fixed, preserve homophily, and so on - so that a perturbation looks "unnoticeable." But none of those heuristics asks the one question that actually defines an adversarial example: did the node's ground-truth label survive the edit?

This gap matters because of a plain fact about real graphs: they are dominated by low-degree nodes. If a node has three neighbors and your budget lets you change three edges, you can detach it from its entire original neighborhood and wire it into a new one. Nothing about the degree distribution needs to look suspicious for the node's true class to have moved. So the paper argues we need a label-aware definition of a semantics-preserving perturbation - one that tracks the ground truth directly, rather than trusting a budget to protect it.

A reference classifier that knows the ground truth

The trouble with tracking ground truth is that on real data we don't have it after a perturbation - nobody labeled the graph you just edited. The paper's key move is to run the controlled experiments on Contextual Stochastic Block Models (CSBMs), a synthetic graph family where the data-generating process is known exactly. On a CSBM you can derive the Bayes optimal classifier g , which by construction outputs the most likely true class given the (possibly perturbed) graph. That g becomes a trusted reference for what the semantics actually are.

With a reference in hand, over-robustness has a crisp definition. Take a model f and a node v . On the clean graph, f and the reference g agree. Now apply a perturbation within the allowed budget. If that perturbation flips g - the true class has genuinely changed - but f keeps its original prediction, then f is over-robust at v : it is holding on to an answer that is now wrong. Figure 2 draws the picture. The model f defends a decision region (the hatched area) that extends past the true boundary drawn by g , and the perturbation balls B around each node reach across that true boundary.

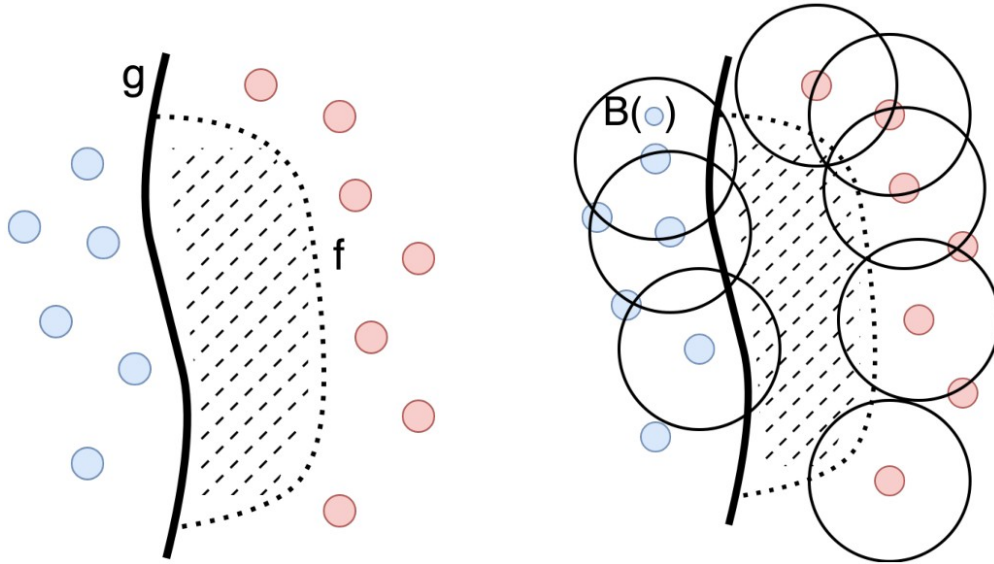


Figure 2. (a) The reference classifier g draws the true decision boundary (solid line); the model f defends a wider region (dotted), and the hatched strip in between is exactly where f stays robust after the semantics have changed. (b) Each node carries a finite perturbation set B ; where those balls cross g 's boundary, a "small" edit has already changed the node's true class.

To turn this into a number, the authors compare two notions of robustness for the same model. $R(f)$ is conventional, degree-corrected robustness: how much perturbation f withstands before changing its prediction, measured against the node's original label. $R(f, g)$ is semantics-aware robustness: the same quantity measured against the reference g , so it only counts robustness up to the point where the semantics actually change. Over-robustness is what conventional robustness claims that the semantics-aware version does not support: $R_{\text{over}} = 1 - R(f, g) / R(f)$. A value of 0.2 means that one fifth of what we counted as the model's robustness actually lies beyond the point of semantic change, and should not have counted as robustness at all.

The experimental setup

The controlled experiments use CSBMs with $n = 1000$ nodes, parametrized to mimic CORA - matching its expected within-class and between-class edge counts ($p = 0.63\%$, $q = 0.15\%$, average degree $d = 21$). A single knob, the signal strength K , controls how discriminative the node features are: at $K = 0.1$ the features are almost noise and the graph structure carries the signal, while at $K = 5$ the features alone suffice and structure becomes unnecessary. For each K the authors sample 10 training graphs, inductively add 1000 fresh test nodes per graph, and average. The synthetic findings are then checked on the real-world Cora-ML graph and on a Contextual Barabasi-Albert model with community structure (CBA).

How much of "robustness" is over-robustness?

The first result is about the threat models themselves, before any model is even blamed. Under a common per-node budget (degree-plus-two edits) at $K = 1.0$, 99.4% of target nodes admit at least one perturbed graph whose true label has changed. In other words, the standard

perturbation sets are saturated with graphs that are not semantics-preserving at all - the very inputs on which we grade a model's robustness.

The second result is about the models. Every architecture tested - GCN, SGC, APPNP, GAT, GATv2, GraphSAGE - is significantly over-robust. To get a ceiling, the authors also measure an MLP, which ignores graph structure entirely and is therefore maximally robust to edge edits; it shows 43% over-robustness at $K = 0.1$ under the weak I2 attack. Every GNN clusters near that upper bound. Figure 3 shows the full sweep: over-robustness R_{over} is high at low K , where structure carries the signal and edits do the most semantic damage, and falls toward zero as K grows and the features alone decide the label.

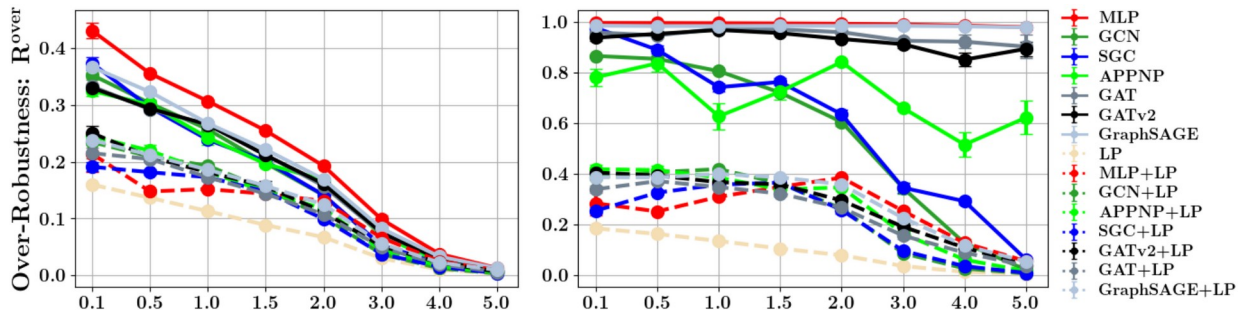


Figure 3. Over-robustness R_{over} versus signal strength K (x-axis runs 0.1 to 5.0). Solid curves are plain models; dashed curves add label propagation (+LP). (a) uses a per-node degree budget; (b) applies the attack with no budget restriction. Adding label propagation (dashed) pulls every curve down, and plain LP (tan) sits lowest of all.

A fix that costs nothing: label propagation

If over-robustness comes from a model clinging to a prediction after the neighborhood has changed, one remedy is to feed the model information it usually ignores at inference time: the labels it already saw during training. The authors combine each GNN's soft predictions with label propagation (label spreading), which diffuses the known training labels across the graph. The dashed curves in Figure 3 are the result - every model's over-robustness drops sharply, and label propagation on its own achieves the lowest over-robustness of any method in the study.

The important detail is that this comes for free. Adding label propagation does not reduce test accuracy, and it often improves genuine adversarial robustness while the graph structure still carries useful signal. Concretely, a GCN combined with label propagation at $K = 0.5$ falls to $R_{\text{over}} = 20.9\%$. The effect survives stronger, targeted attacks too, though it never fully vanishes: under Nettack, a GCN at $K = 0.5$ still shows 11.4% over-robustness, and even a structure-blind MLP at $K = 2$ retains 19.2%. The table below collects the headline numbers.

Table 1. Headline over-robustness numbers. Over-robustness is the fraction of a model's measured robustness that persists past the point of semantic change.

Setting	Value
Nodes with a semantics-changing perturbation ($K = 1.0$, degree-plus-two budget)	99.4%
Over-robustness of a maximally robust classifier ($K = 0.1$, I2-weak)	43%

GCN + label propagation over-robustness ($K = 0.5$)	20.9%
GCN over-robustness under Netattack ($K = 0.5$)	11.4%
MLP over-robustness under Netattack ($K = 2$)	19.2%

Does the synthetic story hold on a real graph?

The CSBM lets the authors compute a ground-truth reference, but the worry is always that synthetic conclusions stay synthetic. Figure 4 addresses this on the real-world Cora-ML graph: it plots how many structure perturbations it takes to change a trained GCN's prediction, grouped by a node's degree. Low-degree nodes flip after only a handful of edits, exactly the regime where a small budget can rewire the whole neighborhood - and precisely the nodes that dominate real graphs. The mechanism the paper isolates on CSBMs is visible on data nobody generated to order.

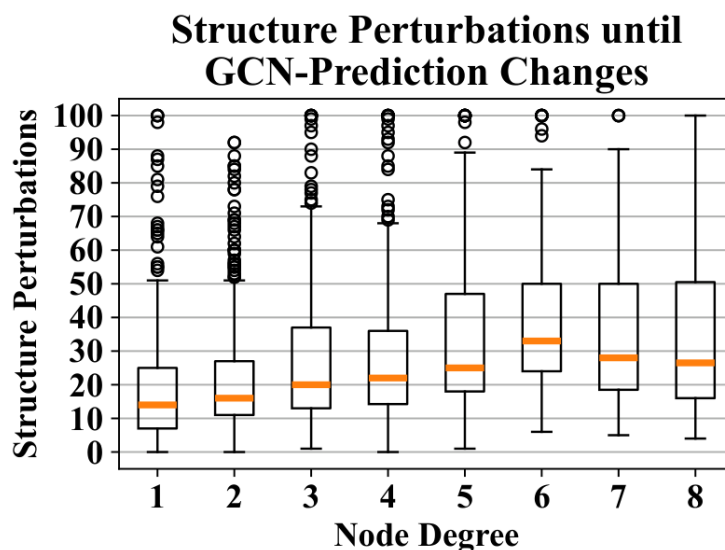


Figure 4. On the real-world Cora-ML graph, the number of structure perturbations needed to flip a GCN's prediction, by node degree. Low-degree nodes - the large majority in real graphs - change after very few edits, the regime where a "small" budget can already alter a node's true class.

Why this reframes robustness for graphs

The takeaway is not that GNNs are secretly fine. It is that our yardstick has been double-counting. A large part of the robustness we measure is over-robustness - a model refusing to update after the node's true meaning has already changed - and conventional evaluations credit that stubbornness as good behavior. Once you measure against the semantics rather than the original label, the picture sharpens: the same edits that look like an attack on a well-defended model are, for many low-degree nodes, legitimate changes of class the model failed to follow.

Two consequences stand out. First, folding the training graph's label structure back into inference through label propagation is a cheap, effective way to cut over-robustness without giving up accuracy or true adversarial robustness. Second, the authors show that under a semantics-aware definition, inductively classifying a newly added node carries no robustness-accuracy tradeoff at all - the tension we thought was fundamental was partly an artifact of

measuring robustness against a label that had already moved. The honest boundary is that all of this rests on having a trustworthy reference for the true class, which CSBMs provide by construction and real graphs do not; extending semantics-aware evaluation to fully real settings is the open problem the paper leaves on the table.

For anyone building or evaluating graph models, the practical message is concrete. Treat a small edit budget as a claim about semantics that needs checking, not a guarantee - especially for the low-degree nodes that make up most of a real graph. Report robustness against a semantics-aware reference wherever one is available, and consider label propagation as a nearly free addition that trims over-robustness without costing accuracy. More broadly, the paper is a reminder that a metric is only as trustworthy as the assumption underneath it - and on graphs, the assumption that a budgeted edit preserves a node's meaning had gone largely unexamined until this work put it to the test.